

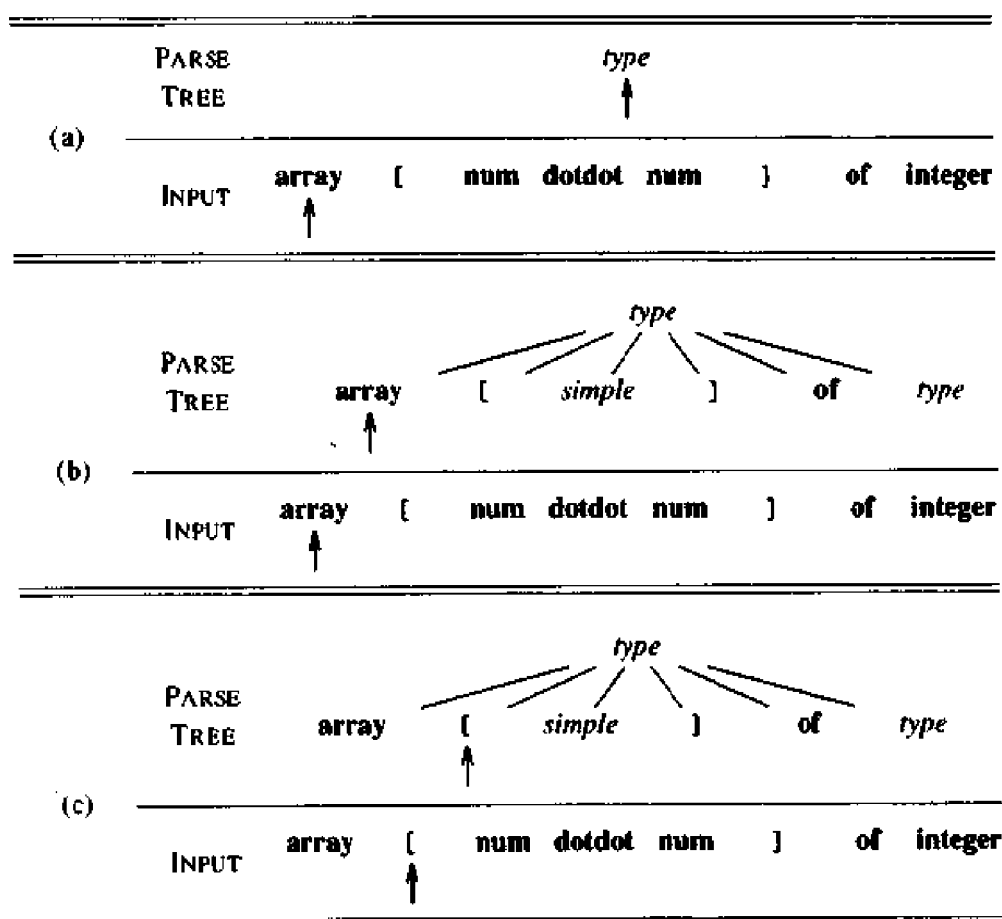


Fig. 2.16. Top-down parsing while scanning the input from left to right.

terminal matches the lookahead symbol, then we advance in both the parse tree and the input. The next token in the input becomes the new lookahead symbol and the next child in the parse tree is considered. In Fig. 2.16(c), the arrow in the parse tree has advanced to the next child of the root and the arrow in the input has advanced to the next token [. After the next advance, the arrow in the parse tree will point to the child labeled with nonterminal *simple*. When a node labeled with a nonterminal is considered, we repeat the process of selecting a production for the nonterminal.

In general, the selection of a production for a nonterminal may involve trial-and-error; that is, we may have to try a production and backtrack to try another production if the first is found to be unsuitable. A production is unsuitable if, after using the production, we cannot complete the tree to match the input string. There is an important special case, however, called predictive parsing, in which backtracking does not occur.

Predictive Parsing

Recursive-descent parsing is a top-down method of syntax analysis in which we execute a set of recursive procedures to process the input. A procedure is associated with each nonterminal of a grammar. Here, we consider a special form of recursive-descent parsing, called *predictive parsing*, in which the lookahead symbol unambiguously determines the procedure selected for each nonterminal. The sequence of procedures called in processing the input implicitly defines a parse tree for the input.

```

procedure match(t: token);
begin
    if lookahead = t then
        lookahead := nexttoken
    else error
end;

procedure type;
begin
    if lookahead is in { integer, char, num } then
        simple
    else if lookahead = '↑' then begin
        match('↑'); match(id)
    end
    else if lookahead = array then begin
        match(array); match('['); simple; match(']'); match(of); type
    end
    else error
end;

procedure simple;
begin
    if lookahead = integer then
        match(integer)
    else if lookahead = char then
        match(char)
    else if lookahead = num then begin
        match(num); match(dotdot); match(num)
    end
    else error
end;

```

Fig. 2.17. Pseudo-code for a predictive parser.

The predictive parser in Fig. 2.17 consists of procedures for the nonterminals *type* and *simple* of grammar (2.8) and an additional procedure *match*. We

use *match* to simplify the code for *type* and *simple*; it advances to the next input token if its argument *t* matches the lookahead symbol. Thus *match* changes the variable *lookahead*, which is the currently scanned input token.

Parsing begins with a call of the procedure for the starting nonterminal *type* in our grammar. With the same input as in Fig. 2.16, *lookahead* is initially the first token **array**. Procedure *type* executes the code

match(**array**); *match*('['); *simple*; *match*(')'); *match*(**of**); *type* (2.9)

corresponding to the right side of the production

type → **array** [*simple*] **of** *type*

Note that each terminal in the right side is matched with the lookahead symbol and that each nonterminal in the right side leads to a call of its procedure.

With the input of Fig. 2.16, after the tokens **array** and **[** are matched, the lookahead symbol is **num**. At this point procedure *simple* is called and the code

match(**num**); *match*(**dotdot**); *match*(**num**)

in its body is executed.

The lookahead symbol guides the selection of the production to be used. If the right side of a production starts with a token, then the production can be used when the lookahead symbol matches the token. Now consider a right side starting with a nonterminal, as in

type → *simple* (2.10)

This production is used if the lookahead symbol can be generated from *simple*. For example, during the execution of the code fragment (2.9), suppose the lookahead symbol is **integer** when control reaches the procedure call *type*. There is no production for *type* that starts with token **integer**. However, a production for *simple* does, so production (2.10) is used by having *type* call procedure *simple* on lookahead **integer**.

Predictive parsing relies on information about what first symbols can be generated by the right side of a production. More precisely, let α be the right side of a production for nonterminal *A*. We define $\text{FIRST}(\alpha)$ to be the set of tokens that appear as the first symbols of one or more strings generated from α . If α is ϵ or can generate ϵ , then ϵ is also in $\text{FIRST}(\alpha)$.² For example,

$\text{FIRST}(\textit{simple}) = \{ \text{integer}, \text{char}, \text{num} \}$
 $\text{FIRST}(\textit{id}) = \{ \text{t} \}$
 $\text{FIRST}(\text{array} [\textit{simple}] \text{of } \textit{type}) = \{ \text{array} \}$

In practice, many production right sides start with tokens, simplifying the

² Productions with ϵ on the right side complicate the determination of the first symbols generated by a nonterminal. For example, if nonterminal *B* can derive the empty string and there is a production $A \rightarrow BC$, then the first symbol generated by *C* can also be the first symbol generated by *A*. If *C* can also generate ϵ , then both $\text{FIRST}(A)$ and $\text{FIRST}(BC)$ contain ϵ .

construction of FIRST sets. An algorithm for computing FIRST's is given in Section 4.4.

The FIRST sets must be considered if there are two productions $A \rightarrow \alpha$ and $A \rightarrow \beta$. Recursive-descent parsing without backtracking requires $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ to be disjoint. The lookahead symbol can then be used to decide which production to use; if the lookahead symbol is in $\text{FIRST}(\alpha)$, then α is used. Otherwise, if the lookahead symbol is in $\text{FIRST}(\beta)$, then β is used.

When to Use ϵ -Productions

Productions with ϵ on the right side require special treatment. The recursive-descent parser will use an ϵ -production as a default when no other production can be used. For example, consider:

$$\begin{aligned} \text{stmt} &\rightarrow \text{begin opt_stmts end} \\ \text{opt_stmts} &\rightarrow \text{stmt_list} \mid \epsilon \end{aligned}$$

While parsing *opt_stmts*, if the lookahead symbol is not in $\text{FIRST}(\text{stmt_list})$, then the ϵ -production is used. This choice is exactly right if the lookahead symbol is **end**. Any lookahead symbol other than **end** will result in an error, detected during the parsing of *stmt*.

Designing a Predictive Parser

A *predictive parser* is a program consisting of a procedure for every nonterminal. Each procedure does two things.

1. It decides which production to use by looking at the lookahead symbol. The production with right side, α is used if the lookahead symbol is in $\text{FIRST}(\alpha)$. If there is a conflict between two right sides for any lookahead symbol, then we cannot use this parsing method on this grammar. A production with ϵ on the right side is used if the lookahead symbol is not in the FIRST set for any other right hand side.
2. The procedure uses a production by mimicking the right side. A nonterminal results in a call to the procedure for the nonterminal, and a token matching the lookahead symbol results in the next input token being read. If at some point the token in the production does not match the lookahead symbol, an error is declared. Figure 2.17 is the result of applying these rules to grammar (2.8).

Just as a translation scheme is formed by extending a grammar, a syntax-directed translator can be formed by extending a predictive parser. An algorithm for this purpose is given in Section 5.5. The following limited construction suffices for the present because the translation schemes implemented in this chapter do not associate attributes with nonterminals:

1. Construct a predictive parser, ignoring the actions in productions.

- Copy the actions from the translation scheme into the parser. If an action appears after grammar symbol X in production p , then it is copied after the code implementing X . Otherwise, if it appears at the beginning of the production, then it is copied just before the code implementing the production.

We shall construct such a translator in the next section.

Left Recursion

It is possible for a recursive-descent parser to loop forever. A problem arises with left-recursive productions like

$$\text{expr} \rightarrow \text{expr} + \text{term}$$

in which the leftmost symbol on the right side is the same as the nonterminal on the left side of the production. Suppose the procedure for *expr* decides to apply this production. The right side begins with *expr* so the procedure for *expr* is called recursively, and the parser loops forever. Note that the look-ahead symbol changes only when a terminal in the right side is matched. Since the production begins with the nonterminal *expr*, no changes to the input take place between recursive calls, causing the infinite loop.

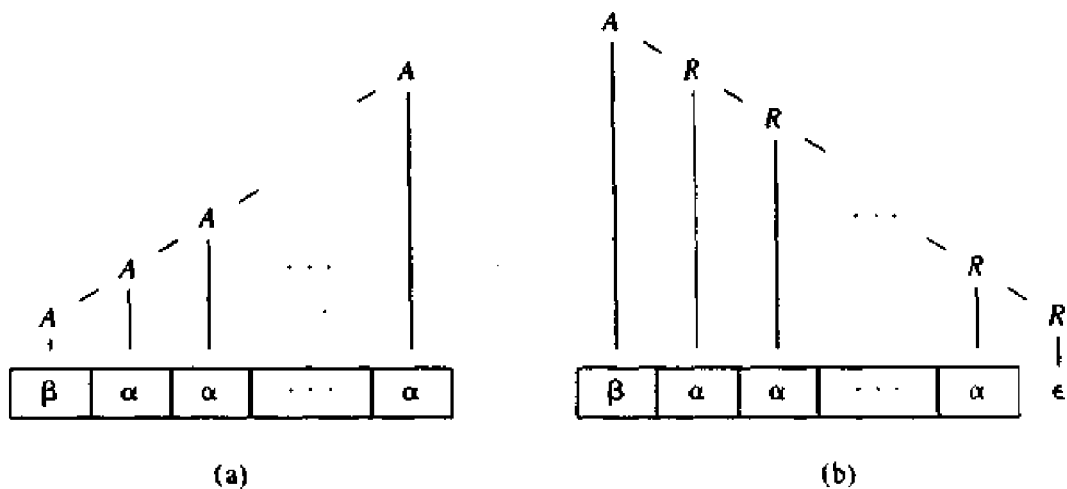


Fig. 2.18. Left- and right-recursive ways of generating a string.

A left-recursive production can be eliminated by rewriting the offending production. Consider a nonterminal A with two productions

$$A \rightarrow A\alpha \mid \beta$$

where α and β are sequences of terminals and nonterminals that do not start with A . For example, in

$$\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$$

$A = \text{expr}$, $\alpha = + \text{term}$, and $\beta = \text{term}$.

The nonterminal A is *left recursive* because the production $A \rightarrow A\alpha$ has A itself as the leftmost symbol on the right side. Repeated application of this production builds up a sequence of α 's to the right of A , as in Fig. 2.18(a). When A is finally replaced by β , we have a β followed by a sequence of zero or more α 's.

The same effect can be achieved, as in Fig. 2.18(b), by rewriting the productions for A in the following manner.

$$\begin{aligned} A &\rightarrow \beta R \\ R &\rightarrow \alpha R \mid \epsilon \end{aligned} \quad (2.11)$$

Here R is a new nonterminal. The production $R \rightarrow \alpha R$ is *right recursive* because this production for R has R itself as the last symbol on the right side. Right-recursive productions lead to trees that grow down towards the right, as in Fig. 2.18(b). Trees growing down to the right make it harder to translate expressions containing left-associative operators, such as minus. In the next section, however, we shall see that the proper translation of expressions into postfix notation can still be attained by a careful design of the translation scheme based on a right-recursive grammar.

In Chapter 4, we consider more general forms of left recursion and show how all left recursion can be eliminated from a grammar.

2.5 A TRANSLATOR FOR SIMPLE EXPRESSIONS

Using the techniques of the last three sections, we now construct a syntax-directed translator, in the form of a working C program, that translates arithmetic expressions into postfix form. To keep the initial program manageably small, we start off with expressions consisting of digits separated by plus and minus signs. The language is extended in the next two sections to include numbers, identifiers, and other operators. Since expressions appear as a construct in so many languages, it is worth studying their translation in detail.

```

expr → expr + term   { print('+') }
expr → expr - term   { print('-') }
expr → term
term → 0              { print('0') }
term → 1              { print('1') }
...
term → 9              { print('9') }
```

Fig. 2.19. Initial specification of infix-to-postfix translator.

A syntax-directed translation scheme can often serve as the specification for a translator. We use the scheme in Fig. 2.19 (repeated from Fig. 2.13) as the

definition of the translation to be performed. As is often the case, the underlying grammar of a given scheme has to be modified before it can be parsed with a predictive parser. In particular, the grammar underlying the scheme in Fig. 2.19 is left-recursive, and as we saw in the last section, a predictive parser cannot handle a left-recursive grammar. By eliminating the left-recursion, we can obtain a grammar suitable for use in a predictive recursive-descent translator.

Abstract and Concrete Syntax

A useful starting point for thinking about the translation of an input string is an *abstract syntax tree* in which each node represents an operator and the children of the node represent the operands. By contrast, a parse tree is called a *concrete syntax tree*, and the underlying grammar is called a *concrete syntax* for the language. Abstract syntax trees, or simply *syntax trees*, differ from parse trees because superficial distinctions of form, unimportant for translation, do not appear in syntax trees.

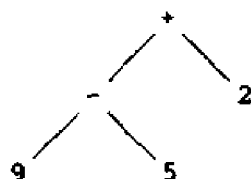


Fig. 2.20. Syntax tree for $9-5+2$.

For example, the syntax tree for $9-5+2$ is shown in Fig. 2.20. Since $+$ and $-$ have the same precedence level, and operators at the same precedence level are evaluated left to right, the tree shows $9-5$ grouped as a subexpression. Comparing Fig. 2.20 with the corresponding parse tree of Fig. 2.2, we note that the syntax tree associates an operator with an interior node, rather than making the operator be one of the children.

It is desirable for a translation scheme to be based on a grammar whose parse trees are as close to syntax trees as possible. The grouping of subexpressions by the grammar in Fig. 2.19 is similar to their grouping in syntax trees. Unfortunately, the grammar of Fig. 2.19 is left-recursive, and hence not suitable for predictive parsing. It appears there is a conflict; on the one hand we need a grammar that facilitates parsing, on the other hand we need a radically different grammar for easy translation. The obvious solution is to eliminate the left-recursion. However, this must be done carefully as the following example shows.

Example 2.9. The following grammar is unsuitable for translating expressions into postfix form, even though it generates exactly the same language as the grammar in Fig. 2.19 and can be used for recursive-descent parsing.

$$\begin{aligned}
 \text{expr} &\rightarrow \text{term rest} \\
 \text{rest} &\rightarrow + \text{expr} \mid - \text{expr} \mid \epsilon \\
 \text{term} &\rightarrow 0 \mid 1 \mid \dots \mid 9
 \end{aligned}$$

This grammar has the problem that the operands of the operators generated by $\text{rest} \rightarrow + \text{expr}$ and $\text{rest} \rightarrow - \text{expr}$ are not obvious from the productions. Neither of the following choices for forming the translation rest.t from that of expr.t is acceptable:

$$\text{rest} \rightarrow - \text{expr} \quad \{ \text{rest.t} := '-' \parallel \text{expr.t} \} \quad (2.12)$$

$$\text{rest} \rightarrow - \text{expr} \quad \{ \text{rest.t} := \text{expr.t} \parallel '-' \} \quad (2.13)$$

(We have only shown the production and semantic action for the minus operator.) The translation of $9-5$ is $95-$. However, if we use the action in (2.12), then the minus sign appears before expr.t and $9-5$ incorrectly remains $9-5$ in translation.

On the other hand, if we use (2.13) and the analogous rule for plus, the operators consistently move to the right end and $9-5+2$ is translated incorrectly into $952+-$ (the correct translation is $95-2+$). $\square$

Adapting the Translation Scheme

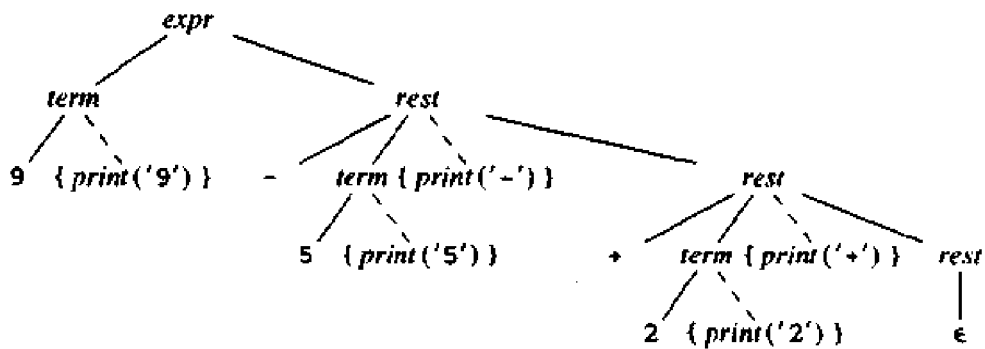
The left-recursion elimination technique sketched in Fig. 2.18 can also be applied to productions containing semantic actions. We extend the transformation in Section 5.5 to take synthesized attributes into account. The technique transforms the productions $A \rightarrow A\alpha \mid A\beta \mid \gamma$ into

$$\begin{aligned}
 A &\rightarrow \gamma R \\
 R &\rightarrow \alpha R \mid \beta R \mid \epsilon
 \end{aligned}$$

When semantic actions are embedded in the productions, we carry them along in the transformation. Here, if we let $A = \text{expr}$, $\alpha = + \text{term} \{ \text{print}('+') \}$, $\beta = - \text{term} \{ \text{print}('-') \}$, and $\gamma = \text{term}$, the transformation above produces the translation scheme (2.14). The expr productions in Fig. 2.19 have been transformed into the productions for expr and the new nonterminal rest in (2.14). The productions for term are repeated from Fig. 2.19. Notice that the underlying grammar is different from the one in Example 2.9 and the difference makes the desired translation possible.

$$\begin{aligned}
 \text{expr} &\rightarrow \text{term rest} \\
 \text{rest} &\rightarrow + \text{term} \{ \text{print}('+') \} \text{rest} \mid - \text{term} \{ \text{print}('-') \} \text{rest} \mid \epsilon \\
 \text{term} &\rightarrow 0 \{ \text{print}('0') \} \\
 \text{term} &\rightarrow 1 \{ \text{print}('1') \} \\
 &\dots \\
 \text{term} &\rightarrow 9 \{ \text{print}('9') \}
 \end{aligned} \quad (2.14)$$

Figure 2.21 shows how $9-5+2$ is translated using the above grammar.

Fig. 2.21. Translation of $9-5+2$ into $95-2+$.

Procedures for the Nonterminals *expr*, *term*, and *rest*

We now implement a translator in C using the syntax-directed translation scheme (2.14). The essence of the translator is the C code in Fig. 2.22 for the functions *expr*, *term*, and *rest*. These functions implement the corresponding nonterminals in (2.14).

```

expr()
{
    term(); rest();
}

rest()
{
    if (lookahead == '+') {
        match('+'); term(); putchar('+'); rest();
    }
    else if (lookahead == '-') {
        match('-'); term(); putchar('-'); rest();
    }
    else ;
}

term()
{
    if (isdigit(lookahead)) {
        putchar(lookahead); match(lookahead);
    }
    else error();
}
  
```

Fig. 2.22. Functions for the nonterminals *expr*, *rest*, and *term*.

The function *match*, presented later, is the C counterpart of the code in

Fig. 2.17 to match a token with the lookahead symbol and advance through the input. Since each token is a single character in our language, match can be implemented by comparing and reading characters.

For those unfamiliar with the programming language C, we mention the salient differences between C and other Algol derivatives such as Pascal, as we find uses for those features of C. A program in C consists of a sequence of function definitions, with execution starting at a distinguished function called *main*. Function definitions cannot be nested. Parentheses enclosing function parameter lists are needed even if there are no parameters; hence we write *expr()*, *term()*, and *rest()*. Functions communicate either by passing parameters "by value" or by accessing data global to all functions. For example, the functions *term()* and *rest()* examine the lookahead symbol using the global identifier *lookahead*.

C and Pascal use the following symbols for assignments and equality tests:

OPERATION	C	PASCAL
assignment	=	:=
equality test	==	=
inequality test	!=	<>

The functions for the nonterminals mimic the right sides of productions. For example, the production *expr* $\rightarrow$ *term rest* is implemented by the calls *term()* and *rest()* in the function *expr()*.

As another example, function *rest()* uses the first production for *rest* in (2.14) if the lookahead symbol is a plus sign, the second production if the lookahead symbol is a minus sign, and the production *rest* $\rightarrow$ ϵ by default. The first production for *rest* is implemented by the first if-statement in Fig. 2.22. If the lookahead symbol is +, the plus sign is matched by the call *match('')*. After the call *term()*, the C standard library routine *putchar('')* implements the semantic action by printing a plus character. Since the third production for *rest* has ϵ as its right side, the last else in *rest()* does nothing.

The ten productions for *term* generate the ten digits. In Fig. 2.22, the routine *isdigit* tests if the lookahead symbol is a digit. The digit is printed and matched if the test succeeds; otherwise, an error occurs. (Note that *match* changes the lookahead symbol, so the printing must occur before the digit is matched.) Before showing a complete program, we shall make one speed-improving transformation to the code in Fig. 2.22.

Optimizing the Translator

Certain recursive calls can be replaced by iterations. When the last statement executed in a procedure body is a recursive call of the same procedure, the call is said to be *tail recursive*. For example, the calls of *rest()* at the end of the fourth and seventh lines of the function *rest()* are tail recursive

because control flows to the end of the function body after each of these calls.

We can speed up a program by replacing tail recursion by iteration. For a procedure without parameters, a tail-recursive call can be simply replaced by a jump to the beginning of the procedure. The code for `rest` can be rewritten as:

```
rest()
{
L:   if (lookahead == '+') {
        match('+'); term(); putchar('+'); goto L;
    }
    else if (lookahead == '-') {
        match('-'); term(); putchar('-'); goto L;
    }
    else ;
}
```

As long as the lookahead symbol is a plus or minus sign, procedure `rest` matches the sign, calls `term` to match a digit, and repeats the process. Note that since `match` removes the sign each time it is called, this cycle occurs only on alternating sequences of signs and digits. If this change is made in Fig. 2.22, the only remaining call of `rest` is from `expr` (see line 3). The two functions can therefore be integrated into one, as shown in Fig. 2.23. In C, a statement *stmt* can be repeatedly executed by writing

```
while(1) stmt
```

because the condition 1 is always true. We can exit from a loop by executing a `break`-statement. The stylized form of the code in Fig. 2.23 allows other operators to be added conveniently.

```
expr()
{
    term();
    while(1)
        if (lookahead == '+') {
            match('+'); term(); putchar('+');
        }
        else if (lookahead == '-') {
            match('-'); term(); putchar('-');
        }
        else break;
}
```

Fig. 2.23. Replacement for functions `expr` and `rest` of Fig. 2.22.

The Complete Program

The complete C program for our translator is shown in Fig. 2.24. The first line, beginning with `#include`, loads `<ctype.h>`, a file of standard routines that contains the code for the predicate `isdigit`.

Tokens, consisting of single characters, are supplied by the standard library routine `getchar` that reads the next character from the input file. However, `lookahead` is declared to be an integer on line 2 of Fig. 2.24 to anticipate the additional tokens that are not single characters that will be introduced in later sections. Since `lookahead` is declared outside any of the functions, it is global to any functions that are defined after line 2 of Fig. 2.24.

The function `match` checks tokens; it reads the next input token if the look-ahead symbol is matched and calls the error routine otherwise.

The function `error` uses the standard library function `printf` to print the message "syntax error" and then terminates execution by the call `exit(1)` to another standard library function.

2.6 LEXICAL ANALYSIS

We shall now add to the translator of the previous section a lexical analyzer that reads and converts the input into a stream of tokens to be analyzed by the parser. Recall from the definition of a grammar in Section 2.2 that the sentences of a language consist of strings of tokens. A sequence of input characters that comprises a single token is called a *lexeme*. A lexical analyzer can insulate a parser from the lexeme representation of tokens. We begin by listing some of the functions we might want a lexical analyzer to perform.

Removal of White Space and Comments

The expression translator in the last section sees every character in the input, so extraneous characters, such as blanks, will cause it to fail. Many languages allow "white space" (blanks, tabs, and newlines) to appear between tokens. Comments can likewise be ignored by the parser and translator, so they may also be treated as white space.

If white space is eliminated by the lexical analyzer, the parser will never have to consider it. The alternative of modifying the grammar to incorporate white space into the syntax is not nearly as easy to implement.

Constants

Anytime a single digit appears in an expression, it seems reasonable to allow an arbitrary integer constant in its place. Since an integer constant is a sequence of digits, integer constants can be allowed either by adding productions to the grammar for expressions, or by creating a token for such constants. The job of collecting digits into integers is generally given to a lexical analyzer because numbers can be treated as single units during translation.

Let `num` be the token representing an integer. When a sequence of digits

```
#include <ctype.h> /* loads file with predicate isdigit */
int lookahead;

main()
{
    lookahead = getchar();
    expr();
    putchar('\n'); /* adds trailing newline character */
}

expr()
{
    term();
    while(1)
        if (lookahead == '+') {
            match('+'); term(); putchar('+');
        }
        else if (lookahead == '-') {
            match('-'); term(); putchar('-');
        }
        else break;
}

term()
{
    if (isdigit(lookahead)) {
        putchar(lookahead);
        match(lookahead);
    }
    else error();
}

match(t)
    int t;
{
    if (lookahead == t)
        lookahead = getchar();
    else error();
}

error()
{
    printf("syntax error\n"); /* print error message */
    exit(1); /* then halt */
}
```

Fig. 2.24. C program to translate an infix expression into postfix form.

appears in the input stream, the lexical analyzer will pass **num** to the parser. The value of the integer will be passed along as an attribute of the token **num**. Logically, the lexical analyzer passes both the token and the attribute to the parser. If we write a token and its attribute as a tuple enclosed between **<>**, the input

31 + 28 + 59

is transformed into the sequence of tuples

<num, 31> **<+, >** **<num, 28>** **<+, >** **<num, 59>**

The token **+** has no attribute. The second components of the tuples, the attributes, play no role during parsing, but are needed during translation.

Recognizing Identifiers and Keywords

Languages use identifiers as names of variables, arrays, functions, and the like. A grammar for a language often treats an identifier as a token. A parser based on such a grammar wants to see the same token, say **id**, each time an identifier appears in the input. For example, the input

count = count + increment; (2.15)

would be converted by the lexical analyzer into the token stream

id = id + id ; (2.16)

This token stream is used for parsing.

When talking about the lexical analysis of the input line (2.15), it is useful to distinguish between the token **id** and the lexemes **count** and **increment** associated with instances of this token. The translator needs to know that the lexeme **count** forms the first two instances of **id** in (2.16) and that the lexeme **increment** forms the third instance of **id**.

When a lexeme forming an identifier is seen in the input, some mechanism is needed to determine if the lexeme has been seen before. As mentioned in Chapter 1, a symbol table is used as such a mechanism. The lexeme is stored in the symbol table and a pointer to this symbol-table entry becomes an attribute of the token **id**.

Many languages use fixed character strings such as **begin**, **end**, **if**, and so on, as punctuation marks or to identify certain constructs. These character strings, called *keywords*, generally satisfy the rules for forming identifiers, so a mechanism is needed for deciding when a lexeme forms a keyword and when it forms an identifier. The problem is easier to resolve if keywords are *reserved*, i.e., if they cannot be used as identifiers. Then a character string forms an identifier only if it is not a keyword.

The problem of isolating tokens also arises if the same characters appear in the lexemes of more than one token, as in **<**, **<=**, and **<>** in Pascal. Techniques for recognizing such tokens efficiently are discussed in Chapter 3.

Interface to the Lexical Analyzer

When a lexical analyzer is inserted between the parser and the input stream, it interacts with the two in the manner shown in Fig. 2.25. It reads characters from the input, groups them into lexemes, and passes the tokens formed by the lexemes, together with their attribute values, to the later stages of the compiler. In some situations, the lexical analyzer has to read some characters ahead before it can decide on the token to be returned to the parser. For example, a lexical analyzer for Pascal must read ahead after it sees the character `>`. If the next character is `=`, then the character sequence `>=` is the lexeme forming the token for the “greater than or equal to” operator. Otherwise `>` is the lexeme forming the “greater than” operator, and the lexical analyzer has read one character too many. The extra character has to be pushed back onto the input, because it can be the beginning of the next lexeme in the input.

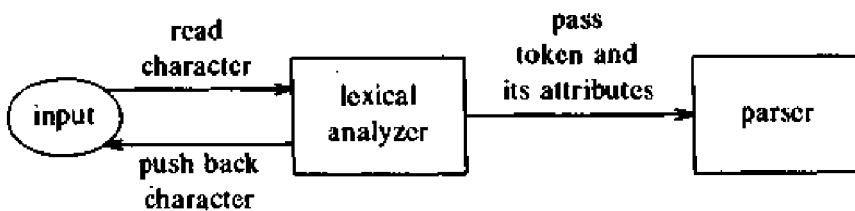


Fig. 2.25. Inserting a lexical analyzer between the input and the parser.

The lexical analyzer and parser form a *producer-consumer* pair. The lexical analyzer produces tokens and the parser consumes them. Produced tokens can be held in a token buffer until they are consumed. The interaction between the two is constrained only by the size of the buffer, because the lexical analyzer cannot proceed when the buffer is full and the parser cannot proceed when the buffer is empty. Commonly, the buffer holds just one token. In this case, the interaction can be implemented simply by making the lexical analyzer be a procedure called by the parser, returning tokens on demand.

The implementation of reading and pushing back characters is usually done by setting up an input buffer. A block of characters is read into the buffer at a time; a pointer keeps track of the portion of the input that has been analyzed. Pushing back a character is implemented by moving back the pointer. Input characters may also need to be saved for error reporting, since some indication has to be given of where in the input text the error occurred. The buffering of input characters can be justified on efficiency grounds alone. Fetching a block of characters is usually more efficient than fetching one character at a time. Techniques for input buffering are discussed in Section 3.2.

A Lexical Analyzer

We now construct a rudimentary lexical analyzer for the expression translator of Section 2.5. The purpose of the lexical analyzer is to allow white space and numbers to appear within expressions. In the next section, we extend the lexical analyzer to allow identifiers as well.

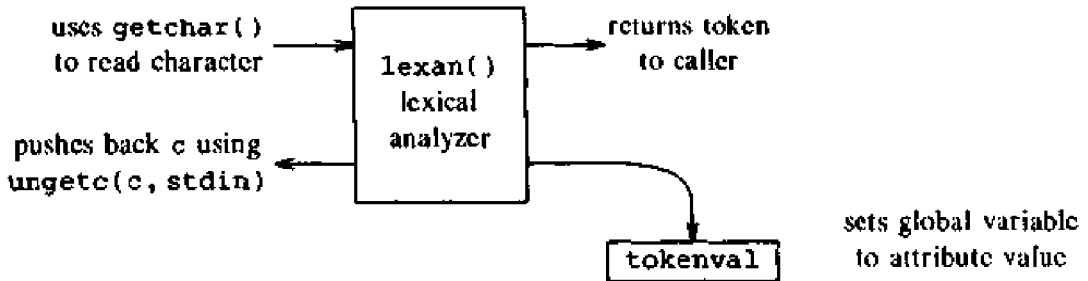


Fig. 2.26. Implementing the interactions in Fig. 2.25.

Figure 2.26 suggests how the lexical analyzer, written as the function `lexan` in C, implements the interactions in Fig. 2.25. The routines `getchar` and `ungetc` from the standard include-file `<stdio.h>` take care of input buffering; `lexan` reads and pushes back input characters by calling the routines `getchar` and `ungetc`, respectively. With `c` declared to be a character, the pair of statements

```
c = getchar(); ungetc(c, stdin);
```

leaves the input stream undisturbed. The call of `getchar` assigns the next input character to `c`; the call of `ungetc` pushes back the value of `c` onto the standard input `stdin`.

If the implementation language does not allow data structures to be returned from functions, then tokens and their attributes have to be passed separately. The function `lexan` returns an integer encoding of a token. The token for a character can be any conventional integer encoding of that character. A token, such as `num`, can then be encoded by an integer larger than any integer encoding a character, say 256. To allow the encoding to be changed easily, we use a symbolic constant `NUM` to refer to the integer encoding of `num`. In Pascal, the association between `NUM` and the encoding can be done by a `const` declaration; in C, `NUM` can be made to stand for 256 using a `define`-statement:

```
#define NUM 256
```

The function `lexan` returns `NUM` when a sequence of digits is seen in the input. A global variable `tokenval` is set to the value of the sequence of digits. Thus, if a 7 is followed immediately by a 6 in the input, `tokenval` is assigned the integer value 76.

Allowing numbers within expressions requires a change in the grammar in Fig. 2.19. We replace the individual digits by the nonterminal *factor* and introduce the following productions and semantic actions:

$$\begin{aligned} \text{factor} &\rightarrow (\text{expr}) \\ &\mid \text{num} \{ \text{print}(\text{num.value}) \} \end{aligned}$$

The C code for *factor* in Fig. 2.27 is a direct implementation of the productions above. When *lookahead* equals NUM, the value of attribute *num.value* is given by the global variable *tokenval*. The action of printing this value is done by the standard library function *printf*. The first argument of *printf* is a string between double quotes specifying the format to be used for printing the remaining arguments. Where *%d* appears in the string, the decimal representation of the next argument is printed. Thus, the *printf* statement in Fig. 2.27 prints a blank followed by the decimal representation of *tokenval* followed by another blank.

```
factor()
{
    if (lookahead == '(') {
        match('('); expr(); match(')');
    }
    else if (lookahead == NUM) {
        printf(" %d ", tokenval); match(NUM);
    }
    else error();
}
```

Fig. 2.27. C code for *factor* when operands can be numbers.

The implementation of function *lexan* is shown in Fig. 2.28. Every time the body of the while statement on lines 8-28 is executed, a character is read into *t* on line 9. If the character is a blank or a tab (written '*\t*'), then no token is returned to the parser; we merely go around the while loop again. If the character is a newline (written '*\n*'), then a global variable *lineno* is incremented, thereby keeping track of line numbers in the input, but again no token is returned. Supplying a line number with an error message helps pinpoint errors.

The code for reading a sequence of digits is on lines 14-23. The predicate *isdigit(t)* from the include-file *<ctype.h>* is used on lines 14 and 17 to determine if an incoming character *t* is a digit. If it is, then its integer value is given by the expression *t-'0'* in both ASCII and EBCDIC. With other character sets, the conversion may need to be done differently. In Section 2.9, we incorporate this lexical analyzer into our expression translator.

```

(1) #include <stdio.h>
(2) #include <ctype.h>
(3) int  lineno = 1;
(4) int  tokenval = NONE;

(5) int  lexan()
(6) {
(7)     int t;
(8)     while(1) {
(9)         t = getchar();
(10)        if (t == ' ' || t == '\t')
(11)            ; /* strip out blanks and tabs */
(12)        else if (t == '\n')
(13)            lineno = lineno + 1;
(14)        else if (isdigit(t)) {
(15)            tokenval = t - '0';
(16)            t = getchar();
(17)            while (isdigit(t)) {
(18)                tokenval = tokenval*10 + t-'0';
(19)                t = getchar();
(20)            }
(21)            ungetc(t, stdin);
(22)            return NUM;
(23)        }
(24)        else {
(25)            tokenval = NONE;
(26)            return t;
(27)        }
(28)    }
(29) }

```

Fig. 2.28. C code for lexical analyzer eliminating white space and collecting numbers.

2.7 INCORPORATING A SYMBOL TABLE

A data structure called a symbol table is generally used to store information about various source language constructs. The information is collected by the analysis phases of the compiler and used by the synthesis phases to generate the target code. For example, during lexical analysis, the character string, or lexeme, forming an identifier is saved in a symbol-table entry. Later phases of the compiler might add to this entry information such as the type of the identifier, its usage (e.g., procedure, variable, or label), and its position in storage. The code generation phase would then use the information to generate the proper code to store and access this variable. In Section 7.6, we discuss the implementation and use of symbol tables in detail. In this section, we

illustrate how the lexical analyzer of the previous section might interact with a symbol table.

The Symbol-Table Interface

The symbol-table routines are concerned primarily with saving and retrieving lexemes. When a lexeme is saved, we also save the token associated with the lexeme. The following operations will be performed on the symbol table.

`insert(s, t)`: Returns index of new entry for string `s`, token `t`.
`lookup(s)`: Returns index of the entry for string `s`,
or 0 if `s` is not found.

The lexical analyzer uses the `lookup` operation to determine whether there is an entry for a lexeme in the symbol table. If no entry exists, then it uses the `insert` operation to create one. We shall discuss an implementation in which the lexical analyzer and parser both know about the format of symbol-table entries.

Handling Reserved Keywords

The symbol-table routines above can handle any collection of reserved keywords. For example, consider tokens `div` and `mod` with lexemes `div` and `mod`, respectively. We can initialize the symbol table using the calls

```
insert("div", div);  
insert("mod", mod);
```

Any subsequent call `lookup("div")` returns the token `div`, so `div` cannot be used as an identifier.

Any collection of reserved keywords can be handled in this way by appropriately initializing the symbol table.

A Symbol-Table Implementation

The data structure for a particular implementation of a symbol table is sketched in Fig. 2.29. We do not wish to set aside a fixed amount of space to hold lexemes forming identifiers; a fixed amount of space may not be large enough to hold a very long identifier and may be wastefully large for a short identifier, such as `i`. In Fig. 2.29, a separate array `lexemes` holds the character string forming an identifier. The string is terminated by an end-of-string character, denoted by `BOS`, that may not appear in identifiers. Each entry in the symbol-table array `symtable` is a record consisting of two fields, `lexptr`, pointing to the beginning of a lexeme, and `token`. Additional fields can hold attribute values, although we shall not do so here.

In Fig. 2.29, the 0th entry is left empty, because `lookup` returns 0 to indicate that there is no entry for a string. The 1st and 2nd entries are for the keywords `div` and `mod`. The 3rd and 4th entries are for identifiers `count` and `i`.

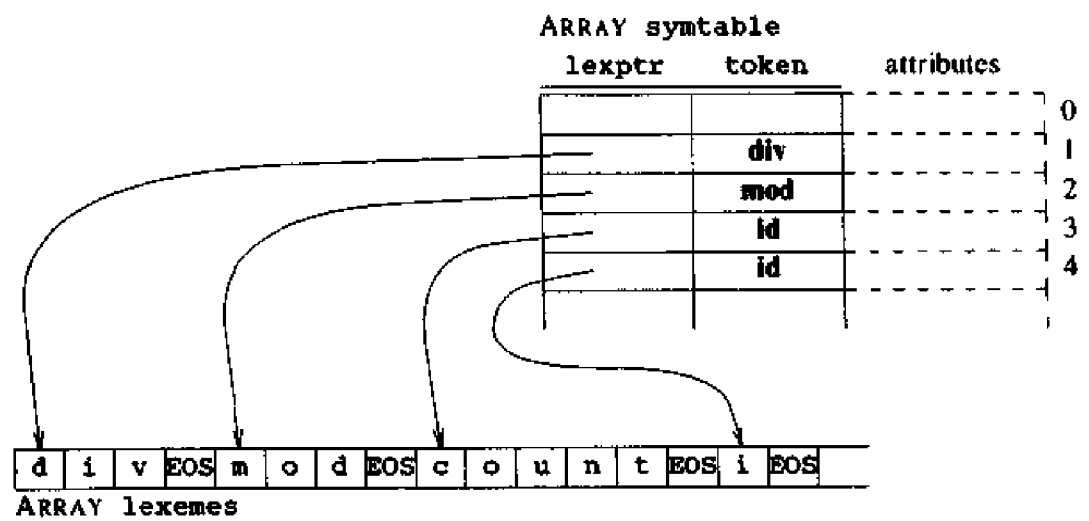


Fig. 2.29. Symbol table and array for storing strings.

Pseudo-code for a lexical analyzer that handles identifiers is shown in Fig. 2.30; a C implementation appears in Section 2.9. White space and integer constants are handled by the lexical analyzer in the same manner as in Fig. 2.28 in the last section.

When our present lexical analyzer reads a letter, it starts saving letters and digits in a buffer `lexbuf`. The string collected in `lexbuf` is then looked up in the symbol table, using the `lookup` operation. Since the symbol table is initialized with entries for the keywords `div` and `mod`, as shown in Fig. 2.29, the lookup operation will find these entries if `lexbuf` contains either `div` or `mod`. If there is no entry for the string in `lexbuf`, i.e., `lookup` returns 0, then `lexbuf` contains a lexeme for a new identifier. An entry for the new identifier is created using `insert`. After the insertion is made, `p` is the index of the symbol-table entry for the string in `lexbuf`. This index is communicated to the parser by setting `tokenval` to `p`, and the token in the token field of the entry is returned.

The default action is to return the integer encoding of the character as a token. Since the single character tokens here have no attributes, `tokenval` is set to `NONE`.

2.8 ABSTRACT STACK MACHINES

The front end of a compiler constructs an intermediate representation of the source program from which the back end generates the target program. One popular form of intermediate representation is code for an abstract stack machine. As mentioned in Chapter 1, partitioning a compiler into a front end and a back end makes it easier to modify a compiler to run on a new machine.

In this section, we present an abstract stack machine and show how code

```

function lexan: integer;
var   lexbuf:   array [0..100] of char;
       c:        char;
begin
    loop begin
        read a character into c;
        if c is a blank or a tab then
            do nothing
        else if c is a newline then
            lineno := lineno + 1
        else if c is a digit then begin
            set tokenval to the value of this and following digits;
            return NUM
        end
        else if c is a letter then begin
            place c and successive letters and digits into lexbuf;
            p := lookup(lexbuf);
            if p = 0 then
                p := insert(lexbuf, ID);
            tokenval := p;
            return the token field of table entry p
        end
        else begin /* token is a single character */
            set tokenval to NONE; /* there is no attribute */
            return integer encoding of character c
        end
    end
end

```

Fig. 2.30. Pseudo-code for a lexical analyzer.

can be generated for it. The machine has separate instruction and data memories and all arithmetic operations are performed on values on a stack. The instructions are quite limited and fall into three classes: integer arithmetic, stack manipulation, and control flow. Figure 2.31 illustrates the machine. The pointer *pc* indicates the instruction we are about to execute. The meanings of the instructions shown will be discussed shortly.

Arithmetic Instructions

The abstract machine must implement each operator in the intermediate language. A basic operation, such as addition or subtraction, is supported directly by the abstract machine. A more complex operation, however, may need to be implemented as a sequence of abstract machine instructions. We simplify the description of the machine by assuming that there is an

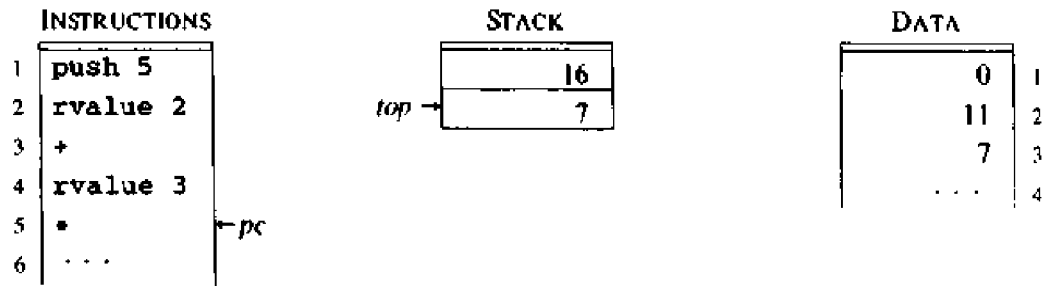


Fig. 2.31. Snapshot of the stack machine after the first four instructions are executed.

instruction for each arithmetic operator.

The abstract machine code for an arithmetic expression simulates the evaluation of a postfix representation for that expression using a stack. The evaluation proceeds by processing the postfix representation from left to right, pushing each operand onto the stack as it is encountered. When a k -ary operator is encountered, its leftmost argument is $k - 1$ positions below the top of the stack and its rightmost argument is at the top. The evaluation applies the operator to the top k values on the stack, pops the operands, and pushes the result onto the stack. For example, in the evaluation of the postfix expression `1 3 + 5 *`, the following actions are performed:

1. Stack 1.
2. Stack 3.
3. Add the two topmost elements, pop them, and stack the result 4.
4. Stack 5.
5. Multiply the two topmost elements, pop them, and stack the result 20.

The value on top of the stack at the end (here 20) is the value of the entire expression.

In the intermediate language, all values will be integers, with 0 corresponding to `false` and nonzero integers corresponding to `true`. The boolean operators `and` and `or` require both their arguments to be evaluated.

L-values and R-values

There is a distinction between the meaning of identifiers on the left and right sides of an assignment. In each of the assignments

```
i := 5;  
i := i + 1;
```

the right side specifies an integer value, while the left side specifies where the value is to be stored. Similarly, if `p` and `q` are pointers to characters, and

```
p↑ := q↑;
```

the right side $q^\dagger$ specifies a character, while $p^\dagger$ specifies where the character is to be stored. The terms *l-value* and *r-value* refer to values that are appropriate on the left and right sides of an assignment, respectively. That is, *r-values* are what we usually think of as "values," while *l-values* are locations.

Stack Manipulation

Besides the obvious instruction for pushing an integer constant onto the stack and popping a value from the top of the stack, there are instructions to access data memory:

push v	push v onto the stack
rvalue l	push contents of data location l
lvalue l	push address of data location l
pop	throw away value on top of the stack
:=	the <i>r-value</i> on top is placed in the <i>l-value</i> below it and both are popped
copy	push a copy of the top value on the stack

Translation of Expressions

Code to evaluate an expression on a stack machine is closely related to postfix notation for that expression. By definition, the postfix form of expression $E + F$ is the concatenation of the postfix form of E , the postfix form of F , and $+$. Similarly, stack-machine code to evaluate $E + F$ is the concatenation of the code to evaluate E , the code to evaluate F , and the instruction to add their values. The translation of expressions into stack-machine code can therefore be done by adapting the translators in Sections 2.6 and 2.7.

Here we generate stack code for expressions in which data locations are addressed symbolically. (The allocation of data locations for identifiers is discussed in Chapter 7.) The expression $a+b$ translates into:

```
rvalue a
rvalue b
+
```

In words: push the contents of the data locations for a and b onto the stack; then pop the top two values on the stack, add them, and push the result onto the stack.

The translation of assignments into stack-machine code is done as follows: the *l-value* of the identifier assigned to is pushed onto the stack, the expression is evaluated, and its *r-value* is assigned to the identifier. For example, the assignment

$$\text{day} := (1461 * y) \text{ div } 4 + (153 * m + 2) \text{ div } 5 + d \quad (2.17)$$

translates into the code in Fig. 2.32.

lvalue day	push 2
push 1461	+
rvalue y	push 5
*	div
push 4	+
div	rvalue d
push 153	+
rvalue m	:=
*	

Fig. 2.32. Translation of $\text{day} := (1461 * y) \text{ div } 4 + (153 * m + 2) \text{ div } 5 + d$.

These remarks can be expressed formally as follows. Each nonterminal has an attribute t giving its translation. Attribute *lexeme* of *id* gives the string representation of the identifier.

$$\begin{aligned} \text{stmt} \rightarrow \text{id} := \text{expr} \\ \{ \text{stmt}.t := \text{'lvalue'} \parallel \text{id.lexeme} \parallel \text{expr}.t \parallel \text{' := '} \} \end{aligned}$$

Control Flow

The stack machine executes instructions in numerical sequence unless told to do otherwise by a conditional or unconditional jump statement. Several options exist for specifying the targets of jumps:

1. The instruction operand gives the target location.
2. The instruction operand specifies the relative distance, positive or negative, to be jumped.
3. The target is specified symbolically; i.e., the machine supports labels.

With the first two options there is the additional possibility of taking the operand from the top of the stack.

We choose the third option for the abstract machine because it is easier to generate such jumps. Moreover, symbolic addresses need not be changed if, after we generate code for the abstract machine, we make certain improvements in the code that result in the insertion or deletion of instructions.

The control-flow instructions for the stack machine are:

label l	target of jumps to l ; has no other effect
goto l	next instruction is taken from statement with label l
gofalse l	pop the top value; jump if it is zero
gotrue l	pop the top value; jump if it is nonzero
halt	stop execution

Translation of Statements

The layout in Fig. 2.33 sketches the abstract-machine code for conditional and while statements. The following discussion concentrates on creating labels.

Consider the code layout for if-statements in Fig. 2.33. There can only be one `label out` instruction in the translation of a source program; otherwise, there will be confusion about where control flows to from a `goto out` statement. We therefore need some mechanism for consistently replacing `out` in the code layout by a unique label every time an if-statement is translated.

Suppose *newlabel* is a procedure that returns a fresh label every time it is called. In the following semantic action, the label returned by a call of *newlabel* is recorded using a local variable *out*:

$$stmt \rightarrow \text{if } expr \text{ then } stmt_1 \quad \{ \begin{array}{l} out := newlabel; \\ stmt.t := expr.t \parallel \\ \text{'gofalse' } out \parallel \\ stmt_1.t \parallel \\ \text{'label' } out \end{array} \right. \quad (2.18)$$

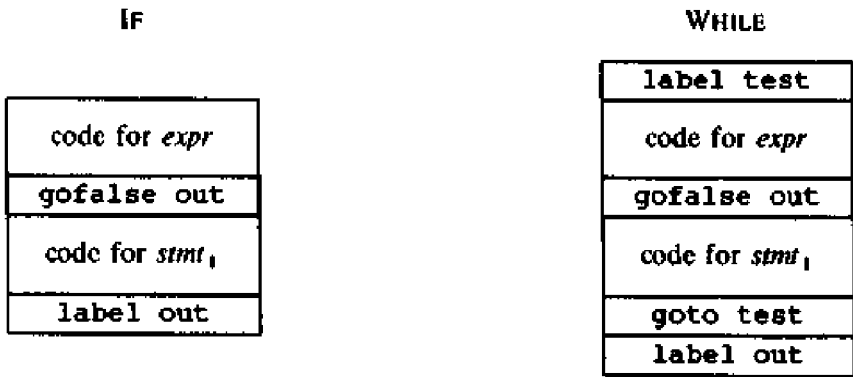


Fig. 2.33. Code layout for conditional and while statements.

Emitting a Translation

The expression translators in Section 2.5 used `print` statements to incrementally generate the translation of an expression. Similar `print` statements can be used to emit the translation of statements. Instead of `print` statements, we use a procedure *emit* to hide printing details. For example, *emit* can worry about whether each abstract-machine instruction needs to be on a separate line. Using the procedure *emit*, we can write the following instead of (2.18):

$$stmt \rightarrow \text{if} \quad \begin{array}{l} expr \quad \{ out := newlabel; emit(\text{'gofalse'}, out); \} \\ \text{then} \\ stmt_1 \quad \{ emit(\text{'label'}, out); \} \end{array}$$

When semantic actions appear within a production, we consider the elements

on the right side of the production in a left-to-right order. For the above production, the order of actions is as follows: actions during the parsing of *expr* are done, *out* is set to the label returned by *newlabel* and the *gofalse* instruction is emitted, actions during the parsing of *stmt₁* are done, and, finally, the *label* instruction is emitted. Assuming the actions during the parsing of *expr* and *stmt₁* emit the code for these nonterminals, the above production implements the code layout of Fig. 2.33.

```

procedure stmt;
var test, out: integer;    /* for labels */
begin
    if lookahead = id then begin
        emit('lvalue', tokenval); match(id); match(':='); expr
    end
    else if lookahead = 'if' then begin
        match('if');
        expr;
        out := newlabel;
        emit('gofalse', out);
        match('then');
        stmt;
        emit('label', out)
    end
    /* code for remaining statements goes here */
    else error;
end

```

Fig. 2.34. Pseudo-code for translating statements.

Pseudo-code for translating assignment and conditional statements is shown in Fig. 2.34. Since variable *out* is local to procedure *stmt*, its value is not affected by the calls to procedures *expr* and *stmt*. The generation of labels requires some thought. Suppose that the labels in the translation are of the form L1, L2, The pseudo-code manipulates such labels using the integer following L. Thus, *out* is declared to be an integer, *newlabel* returns an integer that becomes the value of *out*, and *emit* must be written to print a label given an integer.

The code layout for while statements in Fig. 2.33 can be converted into code in a similar fashion. The translation of a sequence of statements is simply the concatenation of the statements in the sequence, and is left to the reader.

The translation of most single-entry single-exit constructs is similar to that of while statements. We illustrate by considering control flow in expressions.

Example 2.10. The lexical analyzer in Section 2.7 contains a conditional of

the form:

if $t = \text{blank}$ **or** $t = \text{tab}$ **then** . . .

If t is a blank, then clearly it is not necessary to test if t is a tab, because the first equality implies that the condition is true. The expression

expr_1 **or** expr_2

can therefore be implemented as

if expr_1 **then true else** expr_2

The reader can verify that the following code implements the **or** operator:

```
code for  $\text{expr}_1$ 
copy          /* copy value of  $\text{expr}_1$  */
gotrue out
pop           /* pop value of  $\text{expr}_1$  */
code for  $\text{expr}_2$ 
label out
```

Recall that the `gotrue` and `gofalse` instructions pop the value on top of the stack to simplify code generation for conditional and while statements. By copying the value of expr_1 we ensure that the value on top of the stack is true if the `gotrue` instruction leads to a jump. □

2.9 PUTTING THE TECHNIQUES TOGETHER

In this chapter, we have presented a number of syntax-directed techniques for constructing a compiler front end. To summarize these techniques, in this section we put together a C program that functions as an infix-to-postfix translator for a language consisting of sequences of expressions terminated by semicolons. The expressions consist of numbers, identifiers, and the operators `+`, `-`, `*`, `/`, `div`, and `mod`. The output of the translator is a postfix representation for each expression. The translator is an extension of the programs developed in Sections 2.5-2.7. A listing of the complete C program is given at the end of this section.

Description of the Translator

The translator is designed using the syntax-directed translation scheme in Fig. 2.35. The token `id` represents a nonempty sequence of letters and digits beginning with a letter, `num` a sequence of digits, and `eof` an end-of-file character. Tokens are separated by sequences of blanks, tabs, and newlines ("white space"). The attribute *lexeme* of the token `id` gives the character string forming the token; the attribute *value* of the token `num` gives the integer represented by the `num`.

The code for the translator is arranged into seven modules, each stored in a separate file. Execution begins in the module `main.c` that consists of a call

```

start → list eof
list → expr ; list
      | ε
expr → expr + term    { print('+') }
      | expr - term    { print('-') }
      | term
term → term * factor   { print('*') }
      | term / factor   { print('/') }
      | term div factor { print('DIV') }
      | term mod factor { print('MOD') }
      | factor
factor → ( expr )
        | id            { print(id.lexeme) }
        | num           { print(num.value) }

```

Fig. 2.35. Specification for infix-to-postfix translator.

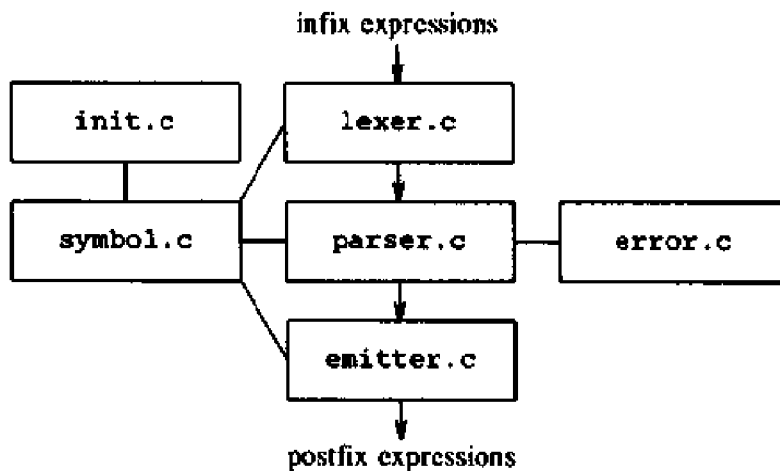


Fig. 2.36. Modules of infix-to-postfix translator.

to `init()` for initialization followed by a call to `parse()` for the translation. The remaining six modules are shown in Fig. 2.36. There is also a global header file `global.h` that contains definitions common to more than one module; the first statement in every module

```
#include "global.h"
```

causes this header file to be included as part of the module. Before showing the code for the translator, we briefly describe each module and how it was constructed.

The Lexical Analysis Module `lexer.c`

The lexical analyzer is a routine called `lexan()` that is called by the parser to find tokens. Implemented from the pseudo-code in Fig. 2.30, the routine reads the input one character at a time and returns to the parser the token it found. The value of the attribute associated with the token is assigned to a global variable `tokenval`.

The following tokens are expected by the parser:

`+ - * / DIV MOD ( ) ID NUM DONE`

Here `ID` represents an identifier, `NUM` a number, and `DONE` the end-of-file character. White space is silently stripped out by the lexical analyzer. The table in Fig. 2.37 shows the token and attribute value produced by the lexical analyzer for each source language lexeme.

LEXEME	TOKEN	ATTRIBUTE VALUE
white space		
sequence of digits	NUM	numeric value of sequence
div.....	DIV	
mod.....	MOD	
other sequences of a letter then letters and digits	ID	index into symtable
end-of-file character	DONE	
any other character	that character	NONE

Fig. 2.37. Description of tokens.

The lexical analyzer uses the symbol-table routine `lookup` to determine whether an identifier lexeme has been previously seen and the routine `insert` to store a new lexeme into the symbol table. It also increments a global variable `lineno` every time it sees a newline character.

The Parser Module `parser.c`

The parser is constructed using the techniques of Section 2.5. We first eliminate left-recursion from the translation scheme of Fig. 2.35 so that the underlying grammar can be parsed with a recursive-descent parser. The transformed scheme is shown in Fig. 2.38.

We then construct functions for the nonterminals *expr*, *term*, and *factor* as we did in Fig. 2.24. The function `parse()` implements the start symbol of the grammar; it calls `lexan` whenever it needs a new token. The parser uses the function `emit` to generate the output and the function `error` to report a syntax error.

```

start → list eof
list → expr ; list
      | ε
expr → term moreterms
moreterms → + term { print('+') } moreterms
           | - term { print('-') } moreterms
           | ε
term → factor morefactors
morefactors → * factor { print('*') } morefactors
            | / factor { print('/') } morefactors
            | div factor { print('DIV') } morefactors
            | mod factor { print('MOD') } morefactors
            | ε
factor → ( expr )
        | id { print(id.lexeme) }
        | num { print(num.value) }

```

Fig. 2.38. Syntax-directed translation scheme after eliminating left-recursion.

The Emitter Module `emitter.c`

The emitter module consists of a single function `emit(t, tval)` that generates the output for token `t` with attribute value `tval`.

The Symbol-Table Modules `symbol.c` and `init.c`

The symbol-table module `symbol.c` implements the data structure shown in Fig. 2.29 of Section 2.7. The entries in the array `syntable` are pairs consisting of a pointer to the `lexemes` array and an integer denoting the token stored there. The operation `insert(s, t)` returns the `syntable` index for the lexeme `s` forming the token `t`. The function `lookup(s)` returns the index of the entry in `syntable` for the lexeme `s` or 0 if `s` is not there.

The module `init.c` is used to preload `syntable` with keywords. The lexeme and token representations for all the keywords are stored in the array `keywords`, which has the same type as the `syntable` array. The function `init()` goes sequentially through the `keyword` array, using the function `insert` to put the keywords in the symbol table. This arrangement allows us to change the representation of the tokens for keywords in a convenient way.

The Error Module `error.c`

The error module manages the error reporting, which is extremely primitive. On encountering a syntax error, the compiler prints a message saying that an error has occurred on the current input line and then halts. A better error recovery technique might skip to the next semicolon and continue parsing; the

reader is encouraged to make this modification to the translator. More sophisticated error recovery techniques are presented in Chapter 4.

Creating the Compiler

The code for the modules appears in seven files: `lexer.c`, `parser.c`, `emitter.c`, `symbol.c`, `init.c`, `error.c`, and `main.c`. The file `main.c` contains the main routine in the C program that calls `init()`, then `parse()`, and upon successful completion `exit(0)`.

Under the UNIX operating system, the compiler can be created by executing the command

```
cc lexer.c parser.c emitter.c symbol.c init.c error.c main.c
```

or by separately compiling the files, using

```
cc -c filename.c
```

and linking the resulting `filename.o` files:

```
cc lexer.o parser.o emitter.o symbol.o init.o error.o main.o
```

The `cc` command creates a file `a.out` that contains the translator. The translator can then be exercised by typing `a.out` followed by the expressions to be translated; e.g.,

```
2+3*5;
12 div 5 mod 2;
```

or whatever other expressions you like. Try it.

The Listing

Here is a listing of the C program implementing the translator. Shown is the global header file `global.h`, followed by the seven source files. For clarity, the program has been written in an elementary C style.

```

/****  global.h  *****/

#include <stdio.h>      /* load i/o routines */
#include <ctype.h>      /* load character test routines */

#define BSIZE 128      /* buffer size */
#define NONE -1
#define EOS '\0'

#define NUM 256
#define DIV 257
#define MOD 258
#define ID 259
#define DONE 260

int tokenval;          /* value of token attribute */

```

```

int lineno;

struct entry {    /* form of symbol table entry */
    char *lexptr;
    int token;
};

struct entry symtable[]; /* symbol table */

/****  lexer.c  *****/

#include "global.h"

char lexbuf[BSIZE];
int lineno = 1;
int tokenval = NONE;

int lexan() /* lexical analyzer */
{
    int t;
    while(1) {
        t = getchar();
        if (t == ' ' || t == '\t')
            ; /* strip out white space */
        else if (t == '\n')
            lineno = lineno + 1;
        else if (isdigit(t)) { /* t is a digit */
            ungetc(t, stdin);
            scanf("%d", &tokenval);
            return NUM;
        }
        else if (isalpha(t)) { /* t is a letter */
            int p, b = 0;
            while (isalnum(t)) { /* t is alphanumeric */
                lexbuf[b] = t;
                t = getchar();
                b = b + 1;
                if (b >= BSIZE)
                    error("compiler error");
            }
            lexbuf[b] = EOS;
            if (t != EOF)
                ungetc(t, stdin);
            p = lookup(lexbuf);
            if (p == 0)
                p = insert(lexbuf, ID);
            tokenval = p;
            return symtable[p].token;
        }
        else if (t == EOF)
            return DONE;
    }
}

```



```

        else {
            tokenval = NONE;
            return t;
        }
    }
}

/****  parser.c  *****/

#include "global.h"

int lookahead;

parse()    /*  parses and translates expression list  */
{
    lookahead = lexan();
    while (lookahead != DONE ) {
        expr(); match(';');
    }
}

expr()
{
    int t;
    term();
    while(1)
        switch (lookahead) {
            case '+': case '-':
                t = lookahead;
                match(lookahead); term(); emit(t, NONE);
                continue;
            default:
                return;
        }
}

term()
{
    int t;
    factor();
    while(1)
        switch (lookahead) {
            case '*': case '/': case DIV: case MOD:
                t = lookahead;
                match(lookahead); factor(); emit(t, NONE);
                continue;
            default:
                return;
        }
}

```

```
factor()
{
    switch(lookahead) {
        case '(':
            match('('); expr(); match(')'); break;
        case NUM:
            emit(NUM, tokenval); match(NUM); break;
        case ID:
            emit(ID, tokenval); match(ID); break;
        default:
            error("syntax error");
    }
}
```

```
match(t)
    int t;
{
    if (lookahead == t)
        lookahead = lexan();
    else error("syntax error");
}
```

/**** emitter.c *****/

```
#include "global.h"
```

```
emit(t, tval)    /* generates output */
    int t, tval;
{
    switch(t) {
        case '+': case '-': case '*': case '/':
            printf("%c\n", t); break;
        case DIV:
            printf("DIV\n"); break;
        case MOD:
            printf("MOD\n"); break;
        case NUM:
            printf("%d\n", tval); break;
        case ID:
            printf("%s\n", symtable[tval].lexptr); break;
        default:
            printf("token %d, tokenval %d\n", t, tval);
    }
}
```

/**** symbol.c *****/

```
#include "global.h"
```

```
#define STRMAX 999    /* size of lexemes array */
#define SYMMAX 100    /* size of symtable */
```

```

char lexemes[STRMAX];
int lastchar = - 1; /* last used position in lexemes */
struct entry symtable[SYMMAX];
int lastentry = 0; /* last used position in symtable */

int lookup(s) /* returns position of entry for s */
char s[];
{
    int p;
    for (p = lastentry; p > 0; p = p - 1)
        if (strcmp(symtable[p].lexptr, s) == 0)
            return p;
    return 0;
}

int insert(s, tok) /* returns position of entry for s */
char s[];
int tok;
{
    int len;
    len = strlen(s); /* strlen computes length of s */
    if (lastentry + 1 >= SYMMAX)
        error("symbol table full");
    if (lastchar + len + 1 >= STRMAX)
        error("lexemes array full");
    lastentry = lastentry + 1;
    symtable[lastentry].token = tok;
    symtable[lastentry].lexptr = &lexemes[lastchar + 1];
    lastchar = lastchar + len + 1;
    strcpy(symtable[lastentry].lexptr, s);
    return lastentry;
}

/**** init.c *****/

#include "global.h"

struct entry keywords[] = {
    "div", DIV,
    "mod", MOD,
    0, 0
};

init() /* loads keywords into symtable */
{
    struct entry *p;
    for (p = keywords; p->token; p++)
        insert(p->lexptr, p->token);
}

```

```

/****  ERROR.C  *****/

#include "global.h"

error(m)      /* generates all error messages */
char *m;
{
    fprintf(stderr, "line %d: %s\n", lineno, m);
    exit(1);    /* unsuccessful termination */
}

/****  main.c  *****/

#include "global.h"

main()
{
    init();
    parse();
    exit(0);    /* successful termination */
}

/*****/

```

EXERCISES

2.1 Consider the context-free grammar

$$S \rightarrow S S + \mid S S * \mid a$$

- Show how the string $aa+a*$ can be generated by this grammar.
- Construct a parse tree for this string.
- What language is generated by this grammar? Justify your answer.

2.2 What language is generated by the following grammars? In each case justify your answer.

- $S \rightarrow 0 S 1 \mid 0 1$
- $S \rightarrow + S S \mid - S S \mid a$
- $S \rightarrow S (S) S \mid \epsilon$
- $S \rightarrow a S b S \mid b S a S \mid \epsilon$
- $S \rightarrow a \mid S + S \mid S S \mid S * \mid (S)$

2.3 Which of the grammars in Exercise 2.2 are ambiguous?

2.4 Construct unambiguous context-free grammars for each of the following languages. In each case show that your grammar is correct.

- a) Arithmetic expressions in postfix notation.
 - b) Left-associative lists of identifiers separated by commas.
 - c) Right-associative lists of identifiers separated by commas.
 - d) Arithmetic expressions of integers and identifiers with the four binary operators $+$, $-$, $*$, $/$.
 - e) Add unary plus and minus to the arithmetic operators of (d).
- *2.5** a) Show that all binary strings generated by the following grammar have values divisible by 3. *Hint.* Use induction on the number of nodes in a parse tree.

$$\text{num} \rightarrow 11 \mid 1001 \mid \text{num } 0 \mid \text{num num}$$

- b) Does the grammar generate all binary strings with values divisible by 3?
- 2.6** Construct a context-free grammar for roman numerals.
- 2.7** Construct a syntax-directed translation scheme that translates arithmetic expressions from infix notation into prefix notation in which an operator appears before its operands; e.g., $-xy$ is the prefix notation for $x-y$. Give annotated parse trees for the inputs $9-5+2$ and $9-5*2$.
- 2.8** Construct a syntax-directed translation scheme that translates arithmetic expressions from postfix notation into infix notation. Give annotated parse trees for the inputs $95-2*$ and $952*-$.
- 2.9** Construct a syntax-directed translation scheme that translates integers into roman numerals.
- 2.10** Construct a syntax-directed translation scheme that translates roman numerals into integers.
- 2.11** Construct recursive-descent parsers for the grammars in Exercise 2.2 (a), (b), and (c).
- 2.12** Construct a syntax-directed translator that verifies that the parentheses in an input string are properly balanced.
- 2.13** The following rules define the translation of an English word into *pig Latin*:
- a) If the word begins with a nonempty string of consonants, move the initial consonant string to the back of the word and add the suffix **AY**; e.g., *pig* becomes *igpay*.
 - b) If the word begins with a vowel, add the suffix **YAY**; e.g., *owl* becomes *owlyay*.
 - c) **U** following a **Q** is a consonant.
 - d) **Y** at the beginning of a word is a vowel if it is not followed by a vowel.

e) One-letter words are not changed.

Construct a syntax-directed translation scheme for pig Latin.

2.14 In the programming language C the for-statement has the form:

for (*expr*₁ ; *expr*₂ ; *expr*₃) *stmt*

The first expression is executed before the loop; it is typically used for initializing the loop index. The second expression is a test made before each iteration of the loop; the loop is exited if the expression becomes 0. The loop itself consists of the statement {*stmt* *expr*₃ ;}. The third expression is executed at the end of each iteration; it is typically used to increment the loop index. The meaning of the for-statement is similar to

*expr*₁ ; **while** (*expr*₂) { *stmt* *expr*₃ ; }

Construct a syntax-directed translation scheme to translate C for-statements into stack-machine code.

***2.15** Consider the following for-statement:

for *i* := 1 **step** 10 - *j* **until** 10 * *j* **do** *j* := *j* + 1

Three semantic definitions can be given for this statement. One possible meaning is that the limit 10 * *j* and increment 10 - *j* are to be evaluated once before the loop, as in PL/I. For example, if *j* = 5 before the loop, we would run through the loop ten times and exit. A second, completely different, meaning would ensue if we are required to evaluate the limit and increment every time through the loop. For example, if *j* = 5 before the loop, the loop would never terminate. A third meaning is given by languages such as Algol. When the increment is negative, the test made for termination of the loop is *i* < 10 * *j*, rather than *i* > 10 * *j*. For each of these three semantic definitions construct a syntax-directed translation scheme to translate these for-loops into stack-machine code.

2.16 Consider the following grammar fragment for if-then- and if-then-else-statements:

```

stmt → if expr then stmt
      | if expr then stmt else stmt
      | other

```

where **other** stands for the other statements in the language.

a) Show that this grammar is ambiguous.

b) Construct an equivalent unambiguous grammar that associates each **else** with the closest previous unmatched **then**.

- c) Construct a syntax-directed translation scheme based on this grammar to translate conditional statements into stack machine code.

***2.17** Construct a syntax-directed translation scheme that translates arithmetic expressions in infix notation into arithmetic expressions in infix notation having no redundant parentheses. Show the annotated parse tree for the input $((1 + 2) * (3 * 4)) + 5$.

PROGRAMMING EXERCISES

- P2.1** Implement a translator from integers to roman numerals based on the syntax-directed translation scheme developed in Exercise 2.9.
- P2.2** Modify the translator in Section 2.9 to produce as output code for the abstract stack machine of Section 2.8.
- P2.3** Modify the error recovery module of the translator in Section 2.9 to skip to the next input expression on encountering an error.
- P2.4** Extend the translator in Section 2.9 to handle all Pascal expressions.
- P2.5** Extend the compiler of Section 2.9 to translate into stack-machine code statements generated by the following grammar:

$$\begin{aligned} \text{stmt} &\rightarrow \text{id} := \text{expr} \\ &\quad | \text{ if } \text{expr} \text{ then } \text{stmt} \\ &\quad | \text{ while } \text{expr} \text{ do } \text{stmt} \\ &\quad | \text{ begin } \text{opt_stmts} \text{ end} \\ \text{opt_stmts} &\rightarrow \text{stmt_list} \quad | \quad \epsilon \\ \text{stmt_list} &\rightarrow \text{stmt_list} ; \text{stmt} \quad | \quad \text{stmt} \end{aligned}$$

- *P2.6** Construct a set of test expressions for the compiler in Section 2.9, so that each production is used at least once in deriving some test expression. Construct a testing program that can be used as a general compiler testing tool. Use your testing program to run your compiler on these test expressions.
- P2.7** Construct a set of test statements for your compiler of Exercise P2.5 so that each production is used at least once to generate some test statement. Use the testing program of Exercise P2.6 to run your compiler on these test expressions.

BIBLIOGRAPHIC NOTES

This introductory chapter touches on a number of subjects that are treated in more detail in the rest of the book. Pointers to the literature appear in the chapters containing further material.

Context-free grammars were introduced by Chomsky [1956] as part of a

study of natural languages. Their use in specifying the syntax of programming languages arose independently. While working with a draft of Algol 60, John Backus "hastily adapted [Emil Post's productions] to that use" (Wexelblat [1981, p.162]). The resulting notation was a variant of context-free grammars. The scholar Panini devised an equivalent syntactic notation to specify the rules of Sanskrit grammar between 400 B.C. and 200 B.C. (Ingerman [1967]).

The proposal that BNF, which began as an abbreviation of Backus Normal Form, be read as Backus-Naur Form, to recognize Naur's contributions as editor of the Algol 60 report (Naur [1963]), is contained in a letter by Knuth [1964].

Syntax-directed definitions are a form of inductive definitions in which the induction is on the syntactic structure. As such they have long been used informally in mathematics. Their application to programming languages came with the use of a grammar to structure the Algol 60 report. Shortly thereafter, Irons [1961] constructed a syntax-directed compiler.

Recursive-descent parsing has been used since the early 1960's. Bauer [1976] attributes the method to Lucas [1961]. Hoare [1962b, p.128] describes an Algol compiler organized as "a set of procedures, each of which is capable of processing one of the syntactic units of the Algol 60 report." Foster [1968] discusses the elimination of left recursion from productions containing semantic actions that do not affect attribute values.

McCarthy [1963] advocated that the translation of a language be based on abstract syntax. In the same paper McCarthy [1963, p.24] left "the reader to convince himself" that a tail-recursive formulation of the factorial function is equivalent to an iterative program.

The benefits of partitioning a compiler into a front end and a back end were explored in a committee report by Strong et al. [1958]. The report coined the name UNCOL (from universal computer oriented language) for a universal intermediate language. The concept has remained an ideal.

A good way to learn about implementation techniques is to read the code of existing compilers. Unfortunately, code is not often published. Randell and Russell [1964] give a comprehensive account of an early Algol compiler. Compiler code may also be seen in McKeeman, Horning, and Wortman [1970]. Barron [1981] is a collection of papers on Pascal implementation, including implementation notes distributed with the Pascal P compiler (Nori et al. [1981]), code generation details (Ammann [1977]), and the code for an implementation of Pascal S, a Pascal subset designed by Wirth [1981] for student use. Knuth [1985] gives an unusually clear and detailed description of the TeX translator.

Kernighan and Pike [1984] describe in detail how to build a desk calculator program around a syntax-directed translation scheme using the compiler-construction tools available on the UNIX operating system. Equation (2.17) is from Tantzen [1963].

CHAPTER 3

Lexical Analysis

This chapter deals with techniques for specifying and implementing lexical analyzers. A simple way to build a lexical analyzer is to construct a diagram that illustrates the structure of the tokens of the source language, and then to hand-translate the diagram into a program for finding tokens. Efficient lexical analyzers can be produced in this manner.

The techniques used to implement lexical analyzers can also be applied to other areas such as query languages and information retrieval systems. In each application, the underlying problem is the specification and design of programs that execute actions triggered by patterns in strings. Since pattern-directed programming is widely useful, we introduce a pattern-action language called Lex for specifying lexical analyzers. In this language, patterns are specified by regular expressions, and a compiler for Lex can generate an efficient finite-automaton recognizer for the regular expressions.

Several other languages use regular expressions to describe patterns. For example, the pattern-scanning language AWK uses regular expressions to select input lines for processing and the UNIX system shell allows a user to refer to a set of file names by writing a regular expression. The UNIX command `rm *.o`, for instance, removes all files with names ending in `".o"`.¹

A software tool that automates the construction of lexical analyzers allows people with different backgrounds to use pattern matching in their own application areas. For example, Jarvis [1976] used a lexical-analyzer generator to create a program that recognizes imperfections in printed circuit boards. The circuits are digitally scanned and converted into "strings" of line segments at different angles. The "lexical analyzer" looked for patterns corresponding to imperfections in the string of line segments. A major advantage of a lexical-analyzer generator is that it can utilize the best-known pattern-matching algorithms and thereby create efficient lexical analyzers for people who are not experts in pattern-matching techniques.

¹ The expression `*.o` is a variant of the usual notation for regular expressions. Exercises 3.10 and 3.14 mention some commonly used variants of regular expression notations.

3.1 THE ROLE OF THE LEXICAL ANALYZER

The lexical analyzer is the first phase of a compiler. Its main task is to read the input characters and produce as output a sequence of tokens that the parser uses for syntax analysis. This interaction, summarized schematically in Fig. 3.1, is commonly implemented by making the lexical analyzer be a subroutine or a coroutine of the parser. Upon receiving a "get next token" command from the parser, the lexical analyzer reads input characters until it can identify the next token.

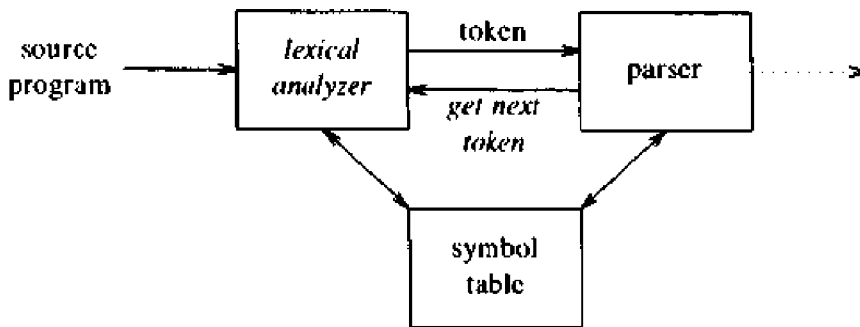


Fig. 3.1. Interaction of lexical analyzer with parser.

Since the lexical analyzer is the part of the compiler that reads the source text, it may also perform certain secondary tasks at the user interface. One such task is stripping out from the source program comments and white space in the form of blank, tab, and newline characters. Another is correlating error messages from the compiler with the source program. For example, the lexical analyzer may keep track of the number of newline characters seen, so that a line number can be associated with an error message. In some compilers, the lexical analyzer is in charge of making a copy of the source program with the error messages marked in it. If the source language supports some macro preprocessor functions, then these preprocessor functions may also be implemented as lexical analysis takes place.

Sometimes, lexical analyzers are divided into a cascade of two phases, the first called "scanning," and the second "lexical analysis." The scanner is responsible for doing simple tasks, while the lexical analyzer proper does the more complex operations. For example, a Fortran compiler might use a scanner to eliminate blanks from the input.

Issues in Lexical Analysis

There are several reasons for separating the analysis phase of compiling into lexical analysis and parsing.

1. Simpler design is perhaps the most important consideration. The separation of lexical analysis from syntax analysis often allows us to simplify

one or the other of these phases. For example, a parser embodying the conventions for comments and white space is significantly more complex than one that can assume comments and white space have already been removed by a lexical analyzer. If we are designing a new language, separating the lexical and syntactic conventions can lead to a cleaner overall language design.

2. Compiler efficiency is improved. A separate lexical analyzer allows us to construct a specialized and potentially more efficient processor for the task. A large amount of time is spent reading the source program and partitioning it into tokens. Specialized buffering techniques for reading input characters and processing tokens can significantly speed up the performance of a compiler.
3. Compiler portability is enhanced. Input alphabet peculiarities and other device-specific anomalies can be restricted to the lexical analyzer. The representation of special or non-standard symbols, such as $\uparrow$ in Pascal, can be isolated in the lexical analyzer.

Specialized tools have been designed to help automate the construction of lexical analyzers and parsers when they are separated. We shall see several examples of such tools in this book.

Tokens, Patterns, Lexemes

When talking about lexical analysis, we use the terms "token," "pattern," and "lexeme" with specific meanings. Examples of their use are shown in Fig. 3.2. In general, there is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a *pattern* associated with the token. The pattern is said to *match* each string in the set. A *lexeme* is a sequence of characters in the source program that is matched by the pattern for a token. For example, in the Pascal statement

```
const pi = 3.1416;
```

the substring `pi` is a lexeme for the token "identifier."

TOKEN	SAMPLE LEXEMES	INFORMAL DESCRIPTION OF PATTERN
const	const	const
if	if	if
relation	<, <=, =, <>, >, >=	< or <= or = or <> or >= or >
id	pi, count, D2	letter followed by letters and digits
num	3.1416, 0, 6.02E23	any numeric constant
literal	"core dumped"	any characters between " and " except "

Fig. 3.2. Examples of tokens.

We treat tokens as terminal symbols in the grammar for the source language, using boldface names to represent tokens. The lexemes matched by the pattern for the token represent strings of characters in the source program that can be treated together as a lexical unit.

In most programming languages, the following constructs are treated as tokens: keywords, operators, identifiers, constants, literal strings, and punctuation symbols such as parentheses, commas, and semicolons. In the example above, when the character sequence `pi` appears in the source program, a token representing an identifier is returned to the parser. The returning of a token is often implemented by passing an integer corresponding to the token. It is this integer that is referred to in Fig. 3.2 as boldface `id`.

A pattern is a rule describing the set of lexemes that can represent a particular token in source programs. The pattern for the token `const` in Fig. 3.2 is just the single string `const` that spells out the keyword. The pattern for the token `relation` is the set of all six Pascal relational operators. To describe precisely the patterns for more complex tokens like `id` (for identifier) and `num` (for number) we shall use the regular-expression notation developed in Section 3.3.

Certain language conventions impact the difficulty of lexical analysis. Languages such as Fortran require certain constructs in fixed positions on the input line. Thus the alignment of a lexeme may be important in determining the correctness of a source program. The trend in modern language design is toward free-format input, allowing constructs to be positioned anywhere on the input line, so this aspect of lexical analysis is becoming less important.

The treatment of blanks varies greatly from language to language. In some languages, such as Fortran or Algol 68, blanks are not significant except in literal strings. They can be added at will to improve the readability of a program. The conventions regarding blanks can greatly complicate the task of identifying tokens.

A popular example that illustrates the potential difficulty of recognizing tokens is the `DO` statement of Fortran. In the statement

```
DO 5 I = 1.25
```

we cannot tell until we have seen the decimal point that `DO` is not a keyword, but rather part of the identifier `DO5I`. On the other hand, in the statement

```
DO 5 I = 1,25
```

we have seven tokens, corresponding to the keyword `DO`, the statement label `5`, the identifier `I`, the operator `=`, the constant `1`, the comma, and the constant `25`. Here, we cannot be sure until we have seen the comma that `DO` is a keyword. To alleviate this uncertainty, Fortran 77 allows an optional comma between the label and index of the `DO` statement. The use of this comma is encouraged because it helps make the `DO` statement clearer and more readable.

In many languages, certain strings are *reserved*; i.e., their meaning is

predefined and cannot be changed by the user. If keywords are not reserved, then the lexical analyzer must distinguish between a keyword and a user-defined identifier. In PL/I, keywords are not reserved; thus, the rules for distinguishing keywords from identifiers are quite complicated as the following PL/I statement illustrates:

```
IF THEN THEN THEN = ELSE; ELSE ELSE = THEN;
```

Attributes for Tokens

When more than one pattern matches a lexeme, the lexical analyzer must provide additional information about the particular lexeme that matched to the subsequent phases of the compiler. For example, the pattern **num** matches both the strings 0 and 1, but it is essential for the code generator to know what string was actually matched.

The lexical analyzer collects information about tokens into their associated attributes. The tokens influence parsing decisions; the attributes influence the translation of tokens. As a practical matter, a token has usually only a single attribute — a pointer to the symbol-table entry in which the information about the token is kept; the pointer becomes the attribute for the token. For diagnostic purposes, we may be interested in both the lexeme for an identifier and the line number on which it was first seen. Both these items of information can be stored in the symbol-table entry for the identifier.

Example 3.1. The tokens and associated attribute-values for the Fortran statement

```
E = M * C ** 2
```

are written below as a sequence of pairs:

<id, pointer to symbol-table entry for E>

<assign_op, >

<id, pointer to symbol-table entry for M>

<mult_op, >

<id, pointer to symbol-table entry for C>

<exp_op, >

<num, integer value 2>

Note that in certain pairs there is no need for an attribute value; the first component is sufficient to identify the lexeme. In this small example, the token **num** has been given an integer-valued attribute. The compiler may store the character string that forms a number in a symbol table and let the attribute of token **num** be a pointer to the table entry. □

Lexical Errors

Few errors are discernible at the lexical level alone, because a lexical analyzer has a very localized view of a source program. If the string `fi` is encountered in a C program for the first time in the context

```
fi ( a == f(x) ) . . .
```

a lexical analyzer cannot tell whether `fi` is a misspelling of the keyword `if` or an undeclared function identifier. Since `fi` is a valid identifier, the lexical analyzer must return the token for an identifier and let some other phase of the compiler handle any error.

But, suppose a situation does arise in which the lexical analyzer is unable to proceed because none of the patterns for tokens matches a prefix of the remaining input. Perhaps the simplest recovery strategy is "panic mode" recovery. We delete successive characters from the remaining input until the lexical analyzer can find a well-formed token. This recovery technique may occasionally confuse the parser, but in an interactive computing environment it may be quite adequate.

Other possible error-recovery actions are:

1. deleting an extraneous character
2. inserting a missing character
3. replacing an incorrect character by a correct character
4. transposing two adjacent characters.

Error transformations like these may be tried in an attempt to repair the input. The simplest such strategy is to see whether a prefix of the remaining input can be transformed into a valid lexeme by just a single error transformation. This strategy assumes most lexical errors are the result of a single error transformation, an assumption usually, but not always, borne out in practice.

One way of finding the errors in a program is to compute the minimum number of error transformations required to transform the erroneous program into one that is syntactically well-formed. We say that the erroneous program has k errors if the shortest sequence of error transformations that will map it into some valid program has length k . Minimum-distance error correction is a convenient theoretical yardstick, but it is not generally used in practice because it is too costly to implement. However, a few experimental compilers have used the minimum-distance criterion to make local corrections.

3.2 INPUT BUFFERING

This section covers some efficiency issues concerned with the buffering of input. We first mention a two-buffer input scheme that is useful when look-ahead on the input is necessary to identify tokens. Then we introduce some useful techniques for speeding up the lexical analyzer, such as the use of "sentinels" to mark the buffer end.

There are three general approaches to the implementation of a lexical analyzer.

1. Use a lexical-analyzer generator, such as the Lex compiler discussed in Section 3.5, to produce the lexical analyzer from a regular-expression-based specification. In this case, the generator provides routines for reading and buffering the input.
2. Write the lexical analyzer in a conventional systems-programming language, using the I/O facilities of that language to read the input.
3. Write the lexical analyzer in assembly language and explicitly manage the reading of input.

The three choices are listed in order of increasing difficulty for the implementor. Unfortunately, the harder-to-implement approaches often yield faster lexical analyzers. Since the lexical analyzer is the only phase of the compiler that reads the source program character-by-character, it is possible to spend a considerable amount of time in the lexical analysis phase, even though the later phases are conceptually more complex. Thus, the speed of lexical analysis is a concern in compiler design. While the bulk of the chapter is devoted to the first approach, the design and use of an automatic generator, we also consider techniques that are helpful in manual design. Section 3.4 discusses transition diagrams, which are a useful concept for the organization of a hand-designed lexical analyzer.

Buffer Pairs

For many source languages, there are times when the lexical analyzer needs to look ahead several characters beyond the lexeme for a pattern before a match can be announced. The lexical analyzers in Chapter 2 used a function `ungetc` to push lookahead characters back into the input stream. Because a large amount of time can be consumed moving characters, specialized buffering techniques have been developed to reduce the amount of overhead required to process an input character. Many buffering schemes can be used, but since the techniques are somewhat dependent on system parameters, we shall only outline the principles behind one class of schemes here.

We use a buffer divided into two N -character halves, as shown in Fig. 3.3. Typically, N is the number of characters on one disk block, e.g., 1024 or 4096.



Fig. 3.3. An input buffer in two halves.

We read N input characters into each half of the buffer with one system read command, rather than invoking a read command for each input character. If fewer than N characters remain in the input, then a special character **eof** is read into the buffer after the input characters, as in Fig. 3.3. That is, **eof** marks the end of the source file and is different from any input character.

Two pointers to the input buffer are maintained. The string of characters between the two pointers is the current lexeme. Initially, both pointers point to the first character of the next lexeme to be found. One, called the forward pointer, scans ahead until a match for a pattern is found. Once the next lexeme is determined, the forward pointer is set to the character at its right end. After the lexeme is processed, both pointers are set to the character immediately past the lexeme. With this scheme, comments and white space can be treated as patterns that yield no token.

If the forward pointer is about to move past the halfway mark, the right half is filled with N new input characters. If the forward pointer is about to move past the right end of the buffer, the left half is filled with N new characters and the forward pointer wraps around to the beginning of the buffer.

This buffering scheme works quite well most of the time, but with it the amount of lookahead is limited, and this limited lookahead may make it impossible to recognize tokens in situations where the distance that the forward pointer must travel is more than the length of the buffer. For example, if we see

```
DECLARE ( ARG1, ARG2, . . . , ARGn )
```

in a PL/I program, we cannot determine whether **DECLARE** is a keyword or an array name until we see the character that follows the right parenthesis. In either case, the lexeme ends at the second **E**, but the amount of lookahead needed is proportional to the number of arguments, which in principle is unbounded.

```
if forward at end of first half then begin
    reload second half;
    forward := forward + 1
end
else if forward at end of second half then begin
    reload first half;
    move forward to beginning of first half
end
else forward := forward + 1;
```

Fig. 3.4. Code to advance forward pointer.

Sentinels

If we use the scheme of Fig. 3.3 exactly as shown, we must check each time we move the forward pointer that we have not moved off one half of the buffer; if we do, then we must reload the other half. That is, our code for advancing the forward pointer performs tests like those shown in Fig. 3.4.

Except at the ends of the buffer halves, the code in Fig. 3.4 requires two tests for each advance of the forward pointer. We can reduce the two tests to one if we extend each buffer half to hold a *sentinel* character at the end. The sentinel is a special character that cannot be part of the source program. A natural choice is `eof`; Fig. 3.5 shows the same buffer arrangement as Fig. 3.3, with the sentinels added.

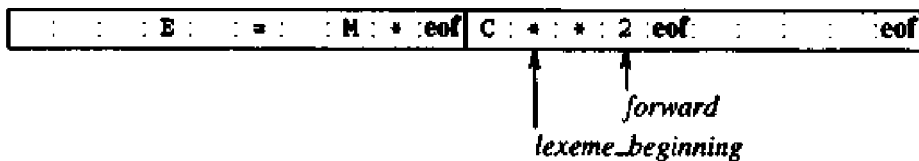


Fig. 3.5. Sentinels at end of each buffer half.

With the arrangement of Fig. 3.5, we can use the code shown in Fig. 3.6 to advance the forward pointer (and test for the end of the source file). Most of the time the code performs only one test to see whether *forward* points to an `eof`. Only when we reach the end of a buffer half or the end of the file do we perform more tests. Since N input characters are encountered between `eof`'s, the average number of tests per input character is very close to 1.

```

forward := forward + 1;
if forward ≠ eof then begin
    if forward at end of first half then begin
        reload second half;
        forward := forward + 1
    end
    else if forward at end of second half then begin
        reload first half;
        move forward to beginning of first half
    end
    else /* eof within a buffer signifying end of input */
        terminate lexical analysis
end
end

```

Fig. 3.6. Lookahead code with sentinels.

We also need to decide how to process the character scanned by the forward pointer; does it mark the end of a token, does it represent progress in finding a particular keyword, or what? One way to structure these tests is to use a case statement, if the implementation language has one. The test

if *forward*† = eof

can then be implemented as one of the different cases.

3.3 SPECIFICATION OF TOKENS

Regular expressions are an important notation for specifying patterns. Each pattern matches a set of strings, so regular expressions will serve as names for sets of strings. Section 3.5 extends this notation into a pattern-directed language for lexical analysis.

Strings and Languages

The term *alphabet* or *character class* denotes any finite set of symbols. Typical examples of symbols are letters and characters. The set $\{0, 1\}$ is the *binary alphabet*. ASCII and EBCDIC are two examples of computer alphabets.

A *string* over some alphabet is a finite sequence of symbols drawn from that alphabet. In language theory, the terms *sentence* and *word* are often used as synonyms for the term "string." The length of a string s , usually written $|s|$, is the number of occurrences of symbols in s . For example, *banana* is a string of length six. The *empty string*, denoted ϵ , is a special string of length zero. Some common terms associated with parts of a string are summarized in Fig. 3.7.

The term *language* denotes any set of strings over some fixed alphabet. This definition is very broad. Abstract languages like $\emptyset$, the *empty set*, or $\{\epsilon\}$, the set containing only the empty string, are languages under this definition. So too are the set of all syntactically well-formed Pascal programs and the set of all grammatically correct English sentences, although the latter two sets are much more difficult to specify. Also note that this definition does not ascribe any meaning to the strings in a language. Methods for ascribing meanings to strings are discussed in Chapter 5.

If x and y are strings, then the *concatenation* of x and y , written xy , is the string formed by appending y to x . For example, if $x = \text{dog}$ and $y = \text{house}$, then $xy = \text{doghouse}$. The empty string is the identity element under concatenation. That is, $s\epsilon = \epsilon s = s$.

If we think of concatenation as a "product", we can define string "exponentiation" as follows. Define s^0 to be ϵ , and for $i > 0$ define s^i to be $s^{i-1}s$. Since ϵs is s itself, $s^1 = s$. Then, $s^2 = ss$, $s^3 = sss$, and so on.